

AI Hands-on: Local AI Tools

Examples and Trade-offs

A comparative overview of three local AI setups for educational use.

Tip: Use [Can I Run It?](#) to check which AI models your machine can actually run locally.

1. Scite.ai MCP Server in LM Studio

About

A research intelligence platform built around *Smart Citations* — a system that classifies how papers cite each other (supporting, contrasting, or mentioning). With 1.6B+ citations from 30+ publisher partnerships, it lets AI tools ground their responses directly in the scientific literature rather than relying on training data alone.

What this example does

LM Studio runs a local LLM with an MCP (Model Context Protocol) server, and the Scite.ai MCP plugin gives the model access to Scite's database of over 1.6 billion citation statements — allowing the LLM to answer questions grounded in peer-reviewed academic literature.

Capabilities:

- Search full-text research across Scite's indexed literature
- Retrieve Smart Citations — supporting, contradicted, or merely mentioned
- Verify factual accuracy of AI-generated claims against real publications
- Connect to institutional holdings for access management

Note: Scite.ai officially supports ChatGPT, Claude, Gemini, and Perplexity. LM Studio works via manual MCP configuration (verified), but the default context window (4096 tokens) must be increased — which increases hardware demand.

✓ Pros

- ✓ **Research-aware responses:** retrieves and cites actual papers with Smart Citations
- ✓ **Local LLM control:** only Scite queries go to the external service
- ✓ **No extra cost for university members:** covered by institutional subscription
- ✓ **Great for academic workflows:** literature-backed answers
- ✓ **1.6B+ citations:** exceptionally broad coverage across disciplines

✗ Cons

- ✗ **LM Studio not officially supported:** requires manual configuration
- ✗ **Context window must be increased:** default 4096 tokens is insufficient
- ✗ **MCP is still emerging:** documentation and community support are limited
- ✗ **High hardware demands:** large context window compounds requirements
- ✗ **Scite queries require internet:** local LLM runs offline, but literature searches don't

2. Ollama + Open Web UI

About

Describes itself as "The freedom AI stack" — a self-hosted, open-source platform (290M+ downloads, 129K GitHub stars). Connects to any AI backend (Ollama, OpenAI, Anthropic, etc.) and includes RAG, voice processing, and image analysis out of the box. Designed around data sovereignty: everything runs on your own infrastructure, with no vendor lock-in.

What this example does

Ollama manages local LLM downloads and serving via a simple CLI, while Open Web UI provides a browser-based chat interface similar to ChatGPT — connecting to Ollama as the backend. Open Web UI is installed and run via **uv** (a fast Python package manager), avoiding the need for Docker.

✓ Pros

- ✓ **Polished user experience:** looks and feels like a modern chat app
- ✓ **Easy model switching:** Llama, Mistral, Gemma, and more directly in the UI
- ✓ **Multi-user support:** conversation history, useful in classroom settings
- ✓ **Fully offline:** no internet connection required after model download
- ✓ **Active community:** well-maintained with extensive documentation

✗ Cons

- ✗ **uv required:** an extra tool participants need to have installed
- ✗ **No native tool/plugin support out of the box:** external tools need extra configuration
- ✗ **Resource heavy:** local model + web server increases RAM and VRAM requirements
- ✗ **No built-in document ingestion:** RAG with your own files requires extra setup

3. Ollama + Open Notebook

About

An open-source, privacy-first research and note-taking platform (MIT-licensed). Rather than a plain chat interface, it acts as a "cognitive partner" — supporting AI-powered note summarization, podcast generation from notes, and content ingestion from PDFs, links, and YouTube videos. Built for researchers, students, and professionals who want full control over their data and model choices.

What this example does

Ollama serves a local LLM, and Open Notebook provides a notebook-style interface — ideal for step-by-step, reproducible AI experiments. Open Notebook is run via **Docker Compose**, which bundles all dependencies into a self-contained environment.

✓ Pros

- ✓ **Research-focused interface:** designed for note-taking, summarization, source management
- ✓ **Rich content ingestion:** PDFs, links, YouTube videos, and text files
- ✓ **Privacy-first:** MIT-licensed, fully self-hosted, you control the data
- ✓ **Podcast generation:** converts notes into audio — distinctive knowledge-sharing feature
- ✓ **Reproducible environment:** Docker Compose ensures consistent setup across machines

✗ Cons

- ✗ **Docker required:** Docker Compose must be installed and running — can be a hurdle for beginners

Summary Comparison

Feature	Scite.ai MCP + LM Studio	Ollama + Open Web UI	Ollama + Open Notebook
Target audience	Researchers, academics	General users, classrooms	Researchers, students, knowledge workers
Technical barrier	Medium	Low–Medium	Low–Medium (Docker required)
Fully offline	Partial (Scite needs internet)	Yes	Yes
Literature integration	Yes (via Scite.ai)	No (out of the box)	Possible (via API calls)
Chat UI	Basic	Polished	Note/research-oriented
Reproducibility	Low	Low	High
Extensibility	Limited (MCP ecosystem)	Moderate	High
Cost	Free for university members	Free	Free